Presentación 1 IA

Nicolás Chehade Casivar
Repo: Link

Temas

- Representación
- Función evaluación
- Lectura
- Greedy propuesto
- Solución inicial con Greedy

Representación

‘Lista con listas de listas, con cada sublista de primer nivel representando una clase, con tres elementos dentro: <Dia, Período, Sala>.

C: C1 y C2
D: Lunes y martes
P: P1 y P2
R: R1
L: C1 requiere 2 clases y C2 requiere 1 clase. (3 ttal)

Una solución sería:
S = [
[(Lunes, P1, R1), (Martes, P2, R1)],
[(Martes, P1, R2)]
]

Tamaño: 3*|L| (+ |C| si se consideran sublistas)
En particular: 9/11

Función de evaluación

Sería la misma función objetivo del modelo planteado:

f. eval: MIN Z = S1+5S2+2S3+S4

Donde:

- S1: Room Capacity
- S2: Minimum Working Days
- S3: Curriculum Compactness
- S4: Room Stability

Lectura

```
./lectura


Nombre instancia:  ToyExample

Cabecera:
  - Courses: 4
  - Rooms: 2
  - Days: 5
  - Periods_per_day: 4
  - Curricula: 2
  - Constraints: 8

COURSES:
  - ID: SceCosc | Teacher: Ocra | #Lectures: 3 | Min. days: 3 | #Students: 30
  - ID: ArcTec | Teacher: Indaco | #Lectures: 3 | Min. days: 2 | #Students: 42
  - ID: TecCos | Teacher: Rosa | #Lectures: 5 | Min. days: 4 | #Students: 40
  - ID: Geotec | Teacher: Scarlatti | #Lectures: 5 | Min. days: 4 | #Students: 18

ROOMS:
  - ID: A | Capacity: 32
  - ID: B | Capacity: 50

CURRICULA:
  - ID: Cur1 | Courses: SceCosc ArcTec TecCos
  - ID: Cur2 | Courses: TecCos Geotec

UNAVAILABILITY_CONSTRAINTS:
  - CourseID: TecCos | Day: 2 | Day_period: 0
  - CourseID: TecCos | Day: 2 | Day_period: 1
  - CourseID: TecCos | Day: 3 | Day_period: 2
  - CourseID: TecCos | Day: 3 | Day_period: 3
  - CourseID: ArcTec | Day: 4 | Day_period: 0
  - CourseID: ArcTec | Day: 4 | Day_period: 1
  - CourseID: ArcTec | Day: 4 | Day_period: 2
  - CourseID: ArcTec | Day: 4 | Day_period: 3

nncc@Nico:/mnt/c/Users/nncc2/OneDrive/Documentos/IA Proyecto/Presentación 1/codes$
```

Greedy propuesto

El algoritmo utiliza la lógica de colocar primero aquellas clases que más restricciones tengan. En particular las restricciones de ‘UNAVAILABILITY_CONSTRAINTS’.

1. Elige el curso con mayor restricciones de disponibilidad. En caso de empate elige aquella con mayor cantidad de lectures. En caso de empate nuevamente elige por orden alfabético por CourseID.
2. Posiciona las lectures de las clases en días, periodos y salas que cumplan con las restricciones.
3. Repetir hasta formar la solución inicial.

Si no consigue construir una solución factible, reinicia el algoritmo cambiando el curso inicial por el siguiente curso según el orden de prioridad. Continúa hasta probar cada curso como punto de partida. Si ninguno permite construir una solución inicial factible, retorna solución vacía.

Solución inicial con greedy

```
Solucion encontrada:

Curso 0:
  Dia: 2 | Periodo: 0 | Sala: B
  Dia: 2 | Periodo: 1 | Sala: B
  Dia: 2 | Periodo: 2 | Sala: A

Curso 1:
  Dia: 1 | Periodo: 1 | Sala: A
  Dia: 1 | Periodo: 2 | Sala: A
  Dia: 1 | Periodo: 3 | Sala: A

Curso 2:
  Dia: 0 | Periodo: 0 | Sala: A
  Dia: 0 | Periodo: 1 | Sala: A
  Dia: 0 | Periodo: 2 | Sala: A
  Dia: 0 | Periodo: 3 | Sala: A
  Dia: 1 | Periodo: 0 | Sala: A

Curso 3:
  Dia: 1 | Periodo: 1 | Sala: B
  Dia: 1 | Periodo: 2 | Sala: B
  Dia: 1 | Periodo: 3 | Sala: B
  Dia: 2 | Periodo: 0 | Sala: A
  Dia: 2 | Periodo: 1 | Sala: A

Verificacion de restricciones:
H1: 1
H2: 1
H3: 1
H4: 1
```